Assignment 8: Hybrid SDN and Wi-Fi Network

Computer Networks Laboratory

Name	Ramnath Thakur	**Roll No.**	123CS0965
Simulator	ns-3.40	**Date**	12 October 2026

1. Aim

Simulate a hybrid SDN and Wi-Fi network in ns-3 and observe how a flooding OpenFlow controller forwards UDP traffic from a mobile station as it moves from one AP to another.

2. Tools

ns-3.40 (openflow, wifi, mobility, csma, bridge, internet, applications, flow-monitor), OpenFlow 0.8.9 switch library, g++ 13.3, Ubuntu 24.04.

3. Theory

- **OpenFlow:** on a table miss the switch sends `PACKET_IN` to the controller, which replies with a `FLOW_MOD` rule. Later packets of that flow are forwarded by the switch directly.
- **Flooding controller:** installs a rule that sends the packet out of every port except the ingress port. No MAC learning. A loop needs a spanning tree, so the S0–S1 link is never flooded.
- **Handover:** the STA drops its AP after missing `MaxMissedBeacons` beacons, then associates with the other AP (same SSID).
- **ConstantVelocityMobilityModel:** `x(t) = 5 + 4t` m.

4. Topology

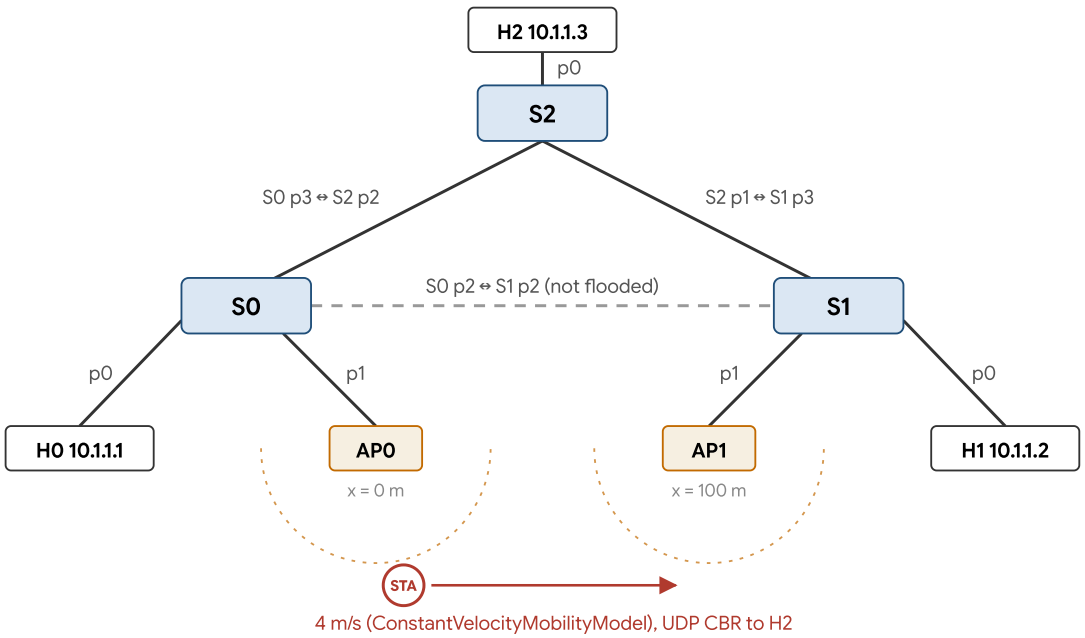
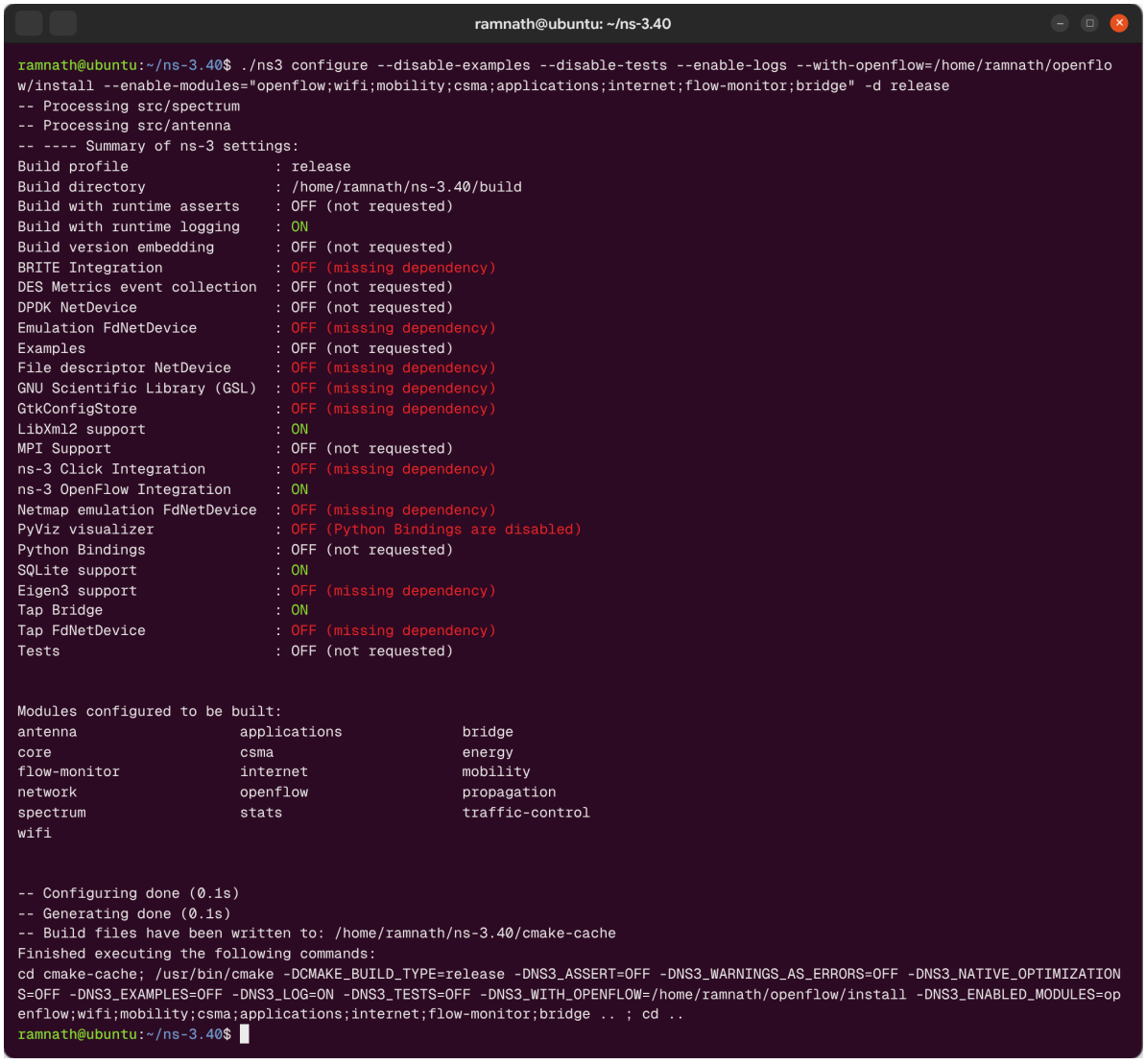


Figure 1. Network topology.

Hosts	H0, H1, H2 = 10.1.1.1–.3, STA = 10.1.1.4
Wired links	CSMA, 100 Mbps, 1 ms
Wi-Fi	802.11g, AP0 at x = 0 m, AP1 at x = 100 m, range 60 m
Mobility	STA from (5, 0) m at 4 m/s along +x
Traffic	UDP CBR STA → H2, 512 B, 400 kbps, 1–29 s
Controller	One `FloodingController` for S0, S1, S2

5. Procedure

1. Build the OpenFlow library and configure ns-3 with `--with-openflow`.
2. Create the switches, hosts, APs and STA, and connect them as in Figure 1.
3. Install OpenFlow switches with the flooding controller.
4. Bridge each AP's Wi-Fi and CSMA interfaces, set mobility, assign IPs.
5. Start UDP traffic from STA to H2, enable traces, run the simulation.



Screenshot 1. Configuring ns-3 with OpenFlow.



Screenshot 2. Building the program.

6. Program

```
#include "ns3/applications-module.h"
#include "ns3/bridge-module.h"
#include "ns3/core-module.h"
#include "ns3/csma-module.h"
#include "ns3/flow-monitor-module.h"
#include "ns3/internet-module.h"
#include "ns3/mobility-module.h"
#include "ns3/network-module.h"
#include "ns3/openflow-module.h"
#include "ns3/wifi-module.h"

#include <fstream>
#include <iomanip>
#include <map>
#include <set>

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("SdnWifiMobility");

static std::ofstream g_flowLog;
static std::ofstream g_eventLog;
static std::ofstream g_rxLog;
static std::ofstream g_tableLog;

static std::map<Ptr<OpenFlowSwitchNetDevice>, std::string> g_swName;

namespace ns3
{
namespace ofi
{
class FloodingController : public Controller
{
  public:
    static TypeId GetTypeId()
    {
        static TypeId tid = TypeId("ns3::ofi::FloodingController")
                                .SetParent<Controller>()
                                .SetGroupName("Openflow")
                                .AddConstructor<FloodingController>();
        return tid;
    }

    void ReceiveFromSwitch(Ptr<OpenFlowSwitchNetDevice> swtch, ofpbuf* buffer) override
    {
        if (m_switches.find(swtch) == m_switches.end())
        {
            return;
        }
        uint8_t type = GetPacketType(buffer);
        if (type == OFPT_PACKET_IN)
        {
            auto opi = (ofp_packet_in*)ofpbuf_try_pull(buffer, offsetof(ofp_packet_in, data));
```

```
            int port = ntohs(opi->in_port);

            sw_flow_key key;
            key.wildcards = 0;
            flow_extract(buffer, port != -1 ? port : OFPP_NONE, &key.flow);

            uint16_t inPort = ntohs(key.flow.in_port);
            std::vector<ofp_action_output> x;
            std::ostringstream outs;
            for (uint32_t p = 0; p < swtch->GetNSwitchPorts(); p++)
            {
                if (p == inPort || m_blocked[swtch].count(p))
                {
                    continue;
                }
                ofp_action_output a;
                memset(&a, 0, sizeof(a));
                a.type = htons(OFPAT_OUTPUT);
                a.len = htons(sizeof(ofp_action_output));
                a.port = p;
                x.push_back(a);
                outs << (outs.tellp() ? " " : "") << p;
            }

            ofp_flow_mod* ofm = BuildFlow(key,
                                          opi->buffer_id,
                                          OFPFC_ADD,
                                          x.data(),
                                          x.size() * sizeof(ofp_action_output),
                                          OFP_FLOW_PERMANENT,
                                          OFP_FLOW_PERMANENT);
            SendToSwitch(swtch, ofm, ofm->header.length);
            m_installed[swtch]++;
            LogFlow(swtch, key, outs.str());
        }
        else if (type == OFPT_FLOW_EXPIRED)
        {
            g_flowLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
                      << "," << g_swName[swtch] << ",EXPIRED,,,,,,," << std::endl;
        }
    }

    void BlockFlood(Ptr<OpenFlowSwitchNetDevice> swtch, uint32_t portNo)
    {
        m_blocked[swtch].insert(portNo);
        NS_LOG_INFO(g_swName[swtch] << " port " << portNo << " excluded from flooding");
    }

    uint32_t Installed(Ptr<OpenFlowSwitchNetDevice> s)
    {
        return m_installed[s];
    }

  private:
    void LogFlow(Ptr<OpenFlowSwitchNetDevice> swtch, const sw_flow_key& key, std::string outs)
    {
        Mac48Address src;
        Mac48Address dst;
        src.CopyFrom(key.flow.dl_src);
        dst.CopyFrom(key.flow.dl_dst);
        uint16_t ethType = ntohs(key.flow.dl_type);
        std::string kind = "OTHER";
        std::ostringstream nw;
        if (ethType == 0x0806)
        {
            kind = "ARP";
        }
        else if (ethType == 0x0800)
        {
            kind = key.flow.nw_proto == 17 ? "UDP" : "IPv4";
```

```
        }
        if (ethType == 0x0800 || ethType == 0x0806)
        {
            nw << Ipv4Address(ntohl(key.flow.nw_src)) << ","
               << Ipv4Address(ntohl(key.flow.nw_dst));
        }
        else
        {
            nw << ",";
        }
        g_flowLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds() << ","
                  << g_swName[swtch] << ",ADD," << ntohs(key.flow.in_port) << "," << kind << ","
                  << src << "," << dst << "," << nw.str() << ",out:" << outs << std::endl;
    }

    std::map<Ptr<OpenFlowSwitchNetDevice>, uint32_t> m_installed;
    std::map<Ptr<OpenFlowSwitchNetDevice>, std::set<uint32_t>> m_blocked;
};

NS_OBJECT_ENSURE_REGISTERED(FloodingController);
}
}

static std::map<Mac48Address, std::string> g_apName;
static uint64_t g_rxBytes = 0;
static uint64_t g_rxPkts = 0;
static const double kSample = 0.5;

static void
OnAssoc(Mac48Address bssid)
{
    g_eventLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
               << ",ASSOC," << g_apName[bssid] << std::endl;
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "s STA associated with " << g_apName[bssid]);
}

static void
OnDeassoc(Mac48Address bssid)
{
    g_eventLog << std::fixed << std::setprecision(3) << Simulator::Now().GetSeconds()
               << ",DEASSOC," << g_apName[bssid] << std::endl;
    NS_LOG_UNCOND(Simulator::Now().GetSeconds() << "s STA lost " << g_apName[bssid]);
}

static void
OnRx(Ptr<const Packet> p)
{
    g_rxBytes += p->GetSize();
    g_rxPkts++;
}

static void
SampleRx(Ptr<MobilityModel> sta)
{
    double kbps = g_rxBytes * 8.0 / kSample / 1000.0;
    g_rxLog << std::fixed << std::setprecision(2) << Simulator::Now().GetSeconds() << "," << kbps
            << "," << g_rxPkts << "," << sta->GetPosition().x << std::endl;
    g_rxBytes = 0;
    g_rxPkts = 0;
    Simulator::Schedule(Seconds(kSample), &SampleRx, sta);
}

static void
SampleTables(std::vector<Ptr<OpenFlowSwitchNetDevice>> sws)
{
    g_tableLog << std::fixed << std::setprecision(1) << Simulator::Now().GetSeconds();
    for (auto& s : sws)
    {
        sw_chain* c = s->GetChain();
        unsigned int n = 0;
```

```
        unsigned long lookups = 0;
        unsigned long hits = 0;
        for (int i = 0; i < c->n_tables; i++)
        {
            sw_table_stats st;
            c->tables[i]->stats(c->tables[i], &st);
            n += st.n_flows;
            lookups += st.n_lookup;
            hits += st.n_matched;
        }
        g_tableLog << "," << n << "," << lookups << "," << hits;
    }
    g_tableLog << std::endl;
    Simulator::Schedule(Seconds(0.5), &SampleTables, sws);
}

int
main(int argc, char* argv[])
{
    double simTime = 30.0;
    double speed = 4.0;
    double apGap = 100.0;
    double range = 60.0;
    std::string rate = "400kbps";
    uint32_t pktSize = 512;
    std::string out = "results";

    CommandLine cmd(__FILE__);
    cmd.AddValue("simTime", "Simulation time (s)", simTime);
    cmd.AddValue("speed", "STA speed (m/s)", speed);
    cmd.AddValue("range", "AP radio range (m)", range);
    cmd.AddValue("out", "Output prefix", out);
    cmd.Parse(argc, argv);

    LogComponentEnable("SdnWifiMobility", LOG_LEVEL_INFO);

    g_flowLog.open(out + "-flows.csv");
    g_flowLog << "time,switch,op,in_port,type,dl_src,dl_dst,nw_src,nw_dst,action" << std::endl;
    g_eventLog.open(out + "-events.csv");
    g_eventLog << "time,event,ap" << std::endl;
    g_rxLog.open(out + "-rx.csv");
    g_rxLog << "time,kbps,pkts,sta_x" << std::endl;
    g_tableLog.open(out + "-tables.csv");
    g_tableLog << "time,S0_flows,S0_lookups,S0_hits,S1_flows,S1_lookups,S1_hits,S2_flows,"
                  "S2_lookups,S2_hits"
               << std::endl;

    NodeContainer switches;
    switches.Create(3);
    NodeContainer hosts;
    hosts.Create(3);
    NodeContainer aps;
    aps.Create(2);
    NodeContainer sta;
    sta.Create(1);

    CsmaHelper csma;
    csma.SetChannelAttribute("DataRate", DataRateValue(DataRate("100Mbps")));
    csma.SetChannelAttribute("Delay", TimeValue(MilliSeconds(1)));

    std::vector<NetDeviceContainer> swPorts(3);
    NetDeviceContainer hostDevs;
    NetDeviceContainer apWired;

    for (uint32_t i = 0; i < 3; i++)
    {
        NetDeviceContainer l = csma.Install(NodeContainer(hosts.Get(i), switches.Get(i)));
        hostDevs.Add(l.Get(0));
        swPorts[i].Add(l.Get(1));
    }
```

```
for (uint32_t i = 0; i < 2; i++)
{
    NetDeviceContainer l = csma.Install(NodeContainer(aps.Get(i), switches.Get(i)));
    apWired.Add(l.Get(0));
    swPorts[i].Add(l.Get(1));
}

uint32_t pairs[3][2] = {{0, 1}, {1, 2}, {2, 0}};
std::map<std::pair<uint32_t, uint32_t>, uint32_t> portOf;
for (auto& pr : pairs)
{
    NetDeviceContainer l = csma.Install(NodeContainer(switches.Get(pr[0]), switches.Get(pr[1])));
    portOf[{pr[0], pr[1]}] = swPorts[pr[0]].GetN();
    swPorts[pr[0]].Add(l.Get(0));
    portOf[{pr[1], pr[0]}] = swPorts[pr[1]].GetN();
    swPorts[pr[1]].Add(l.Get(1));
}

Ptr<ofi::FloodingController> ctrl = CreateObject<ofi::FloodingController>();
OpenFlowSwitchHelper ofHelper;
std::vector<Ptr<OpenFlowSwitchNetDevice>> ofDevs;
for (uint32_t i = 0; i < 3; i++)
{
    NetDeviceContainer d = ofHelper.Install(switches.Get(i), swPorts[i], ctrl);
    Ptr<OpenFlowSwitchNetDevice> of = DynamicCast<OpenFlowSwitchNetDevice>(d.Get(0));
    g_swName[of] = "S" + std::to_string(i);
    ofDevs.push_back(of);
}

ctrl->BlockFlood(ofDevs[0], portOf[{0, 1}]);
ctrl->BlockFlood(ofDevs[1], portOf[{1, 0}]);

YansWifiChannelHelper chan;
chan.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
chan.AddPropagationLoss("ns3::RangePropagationLossModel", "MaxRange", DoubleValue(range));
YansWifiPhyHelper phy;
phy.SetChannel(chan.Create());

WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211g);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
                             "DataMode",
                             StringValue("ErpOfdmRate24Mbps"),
                             "ControlMode",
                             StringValue("ErpOfdmRate6Mbps"));
WifiMacHelper mac;
Ssid ssid("sdn-lab");

mac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
NetDeviceContainer apWifi = wifi.Install(phy, mac, aps);

mac.SetType("ns3::StaWifiMac",
            "Ssid",
            SsidValue(ssid),
            "ActiveProbing",
            BooleanValue(false),
            "MaxMissedBeacons",
            UintegerValue(5));
NetDeviceContainer staWifi = wifi.Install(phy, mac, sta);

BridgeHelper bridge;
for (uint32_t i = 0; i < 2; i++)
{
    NetDeviceContainer b;
    b.Add(apWifi.Get(i));
    b.Add(apWired.Get(i));
    bridge.Install(aps.Get(i), b);
    g_apName[Mac48Address::ConvertFrom(apWifi.Get(i)->GetAddress())] = "AP" + std::to_string(i);
}
```

```
    MobilityHelper fixed;
    Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
    pos->Add(Vector(0, 0, 0));
    pos->Add(Vector(apGap, 0, 0));
    fixed.SetPositionAllocator(pos);
    fixed.SetMobilityModel("ns3::ConstantPositionMobilityModel");
    fixed.Install(aps);

    MobilityHelper moving;
    moving.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
    moving.Install(sta);
    Ptr<ConstantVelocityMobilityModel> staMob = sta.Get(0)->GetObject<ConstantVelocityMobilityModel>();
    staMob->SetPosition(Vector(5, 0, 0));
    staMob->SetVelocity(Vector(speed, 0, 0));

    InternetStackHelper stack;
    stack.Install(hosts);
    stack.Install(sta);
    Ipv4AddressHelper ip;
    ip.SetBase("10.1.1.0", "255.255.255.0");
    Ipv4InterfaceContainer hostIf = ip.Assign(hostDevs);
    Ipv4InterfaceContainer staIf = ip.Assign(staWifi);

    uint16_t port = 9000;
    UdpServerHelper server(port);
    ApplicationContainer srv = server.Install(hosts.Get(2));
    srv.Start(Seconds(0.5));
    srv.Stop(Seconds(simTime));

    UdpClientHelper client(hostIf.GetAddress(2), port);
    double interval = pktSize * 8.0 / DataRate(rate).GetBitRate();
    client.SetAttribute("Interval", TimeValue(Seconds(interval)));
    client.SetAttribute("PacketSize", UintegerValue(pktSize));
    client.SetAttribute("MaxPackets", UintegerValue(1000000));
    ApplicationContainer cli = client.Install(sta.Get(0));
    cli.Start(Seconds(1.0));
    cli.Stop(Seconds(simTime - 1));

    srv.Get(0)->TraceConnectWithoutContext("Rx", MakeCallback(&OnRx));


Config::ConnectWithoutContext("/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/Mac/$ns3::StaWifiMac/Assoc",
                              MakeCallback(&OnAssoc));

Config::ConnectWithoutContext("/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/Mac/$ns3::StaWifiMac/DeAssoc",
                              MakeCallback(&OnDeassoc));
    Simulator::Schedule(Seconds(kSample), &SampleRx, staMob);
    Simulator::Schedule(Seconds(0.5), &SampleTables, ofDevs);

    csma.EnablePcap(out + "-h2", hostDevs.Get(2), true);

    FlowMonitorHelper fmh;
    Ptr<FlowMonitor> fm = fmh.InstallAll();

    Simulator::Stop(Seconds(simTime));
    Simulator::Run();

    Ptr<UdpServer> us = DynamicCast<UdpServer>(srv.Get(0));
    Ptr<UdpClient> uc = DynamicCast<UdpClient>(cli.Get(0));
    std::cout << "\n=== UDP summary ===\n"
              << "Received at H2      : " << us->GetReceived() << "\n"
              << "Lost (seq gaps)     : " << us->GetLost() << "\n";

    fm->CheckForLostPackets();
    auto cls = DynamicCast<Ipv4FlowClassifier>(fmh.GetClassifier());
    std::ofstream fmOut(out + "-flowmon.csv");
    fmOut << "flow,src,dst,proto,tx,rx,lost,throughput_kbps,mean_delay_ms,mean_jitter_ms\n";
    for (auto& kv : fm->GetFlowStats())
    {
```

```
        auto t = cls->FindFlow(kv.first);
        auto& s = kv.second;
        double dur = (s.timeLastRxPacket - s.timeFirstTxPacket).GetSeconds();
        double thr = dur > 0 ? s.rxBytes * 8.0 / dur / 1000.0 : 0;
        double d = s.rxPackets ? s.delaySum.GetSeconds() * 1000 / s.rxPackets : 0;
        double j = s.rxPackets > 1 ? s.jitterSum.GetSeconds() * 1000 / (s.rxPackets - 1) : 0;
        fmOut << kv.first << "," << t.sourceAddress << "," << t.destinationAddress << ","
              << (int)t.protocol << "," << s.txPackets << "," << s.rxPackets << ","
              << s.txPackets - s.rxPackets << "," << thr << "," << d << "," << j << "\n";
        std::cout << "Flow " << kv.first << " " << t.sourceAddress << " -> "
                  << t.destinationAddress << "  tx=" << s.txPackets << " rx=" << s.rxPackets
                  << " thr=" << thr << " kbps delay=" << d << " ms\n";
    }
    for (uint32_t i = 0; i < 3; i++)
    {
        std::cout << "S" << i << " flow_mods installed: " << ctrl->Installed(ofDevs[i]) << "\n";
    }

    Simulator::Destroy();
    return 0;
}
```

7. Output

```
ramnath@ubuntu: ~/ns-3.40

ramnath@ubuntu:~/ns-3.40$ ./ns3 run "sdn-wifi-mobility --out=results"

[0/2] Re-checking globbed directories...
ninja: no work to do.
S0 port 2 excluded from flooding
S1 port 2 excluded from flooding
0.12161s STA associated with AP0
14.222s STA lost AP0
14.3436s STA associated with AP1

=== UDP summary ===
Received at H2      : 2677
Lost (seq gaps)     : 58
Flow 1 10.1.1.4 -> 10.1.1.3  tx=2735 rx=2677 thr=413.03 kbps delay=3.39491 ms
S0 flow_mods installed: 5
S1 flow_mods installed: 5
S2 flow_mods installed: 5
ramnath@ubuntu:~/ns-3.40$
```

Screenshot 3. Simulation output.

```
ramnath@ubuntu: ~/ns-3.40

ramnath@ubuntu:~/ns-3.40$ du -h results-*
4.0K    results-events.csv
4.0K    results-flowmon.csv
4.0K    results-flows.csv
1.5M    results-h2-5-0.pcap
4.0K    results-rx.csv
4.0K    results-tables.csv
ramnath@ubuntu:~/ns-3.40$ column -s, -t < results-flows.csv
time    switch  op   in_port  type  dl_src             dl_dst             nw_src    nw_dst    action
1.010   S0      ADD  1        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 3
1.011   S2      ADD  2        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 1
1.012   S1      ADD  3        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 1
1.013   S2      ADD  0        ARP   00:00:00:00:00:05  00:00:00:00:00:13  0.0.0.0   0.0.0.0   out:1 2
1.014   S1      ADD  3        ARP   00:00:00:00:00:05  00:00:00:00:00:13  0.0.0.0   0.0.0.0   out:0 1
1.014   S0      ADD  3        ARP   00:00:00:00:00:05  00:00:00:00:00:13  0.0.0.0   0.0.0.0   out:0 1
1.017   S0      ADD  1        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 3
1.018   S2      ADD  2        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 1
1.019   S1      ADD  3        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 1
14.356  S1      ADD  1        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 3
14.357  S2      ADD  1        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 2
14.358  S0      ADD  3        ARP   00:00:00:00:00:13  ff:ff:ff:ff:ff:ff  0.0.0.0   0.0.0.0   out:0 1
14.362  S1      ADD  1        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 3
14.363  S2      ADD  1        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 2
14.364  S0      ADD  3        UDP   00:00:00:00:00:13  00:00:00:00:00:05  10.1.1.4  10.1.1.3  out:0 1
ramnath@ubuntu:~/ns-3.40$
```

Screenshot 4. Flows installed by the controller.

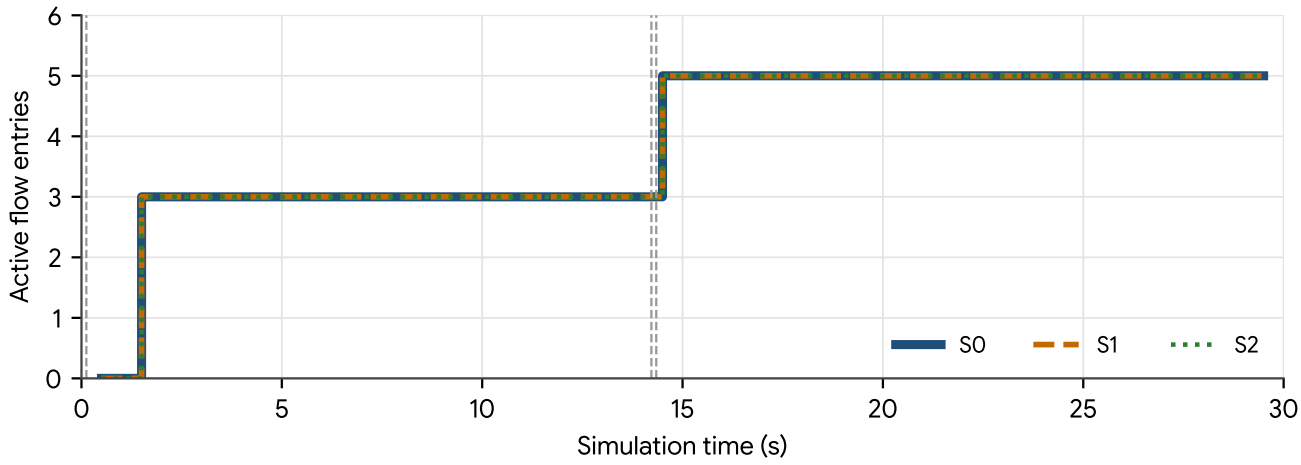


Figure 2. Flow entries per switch.

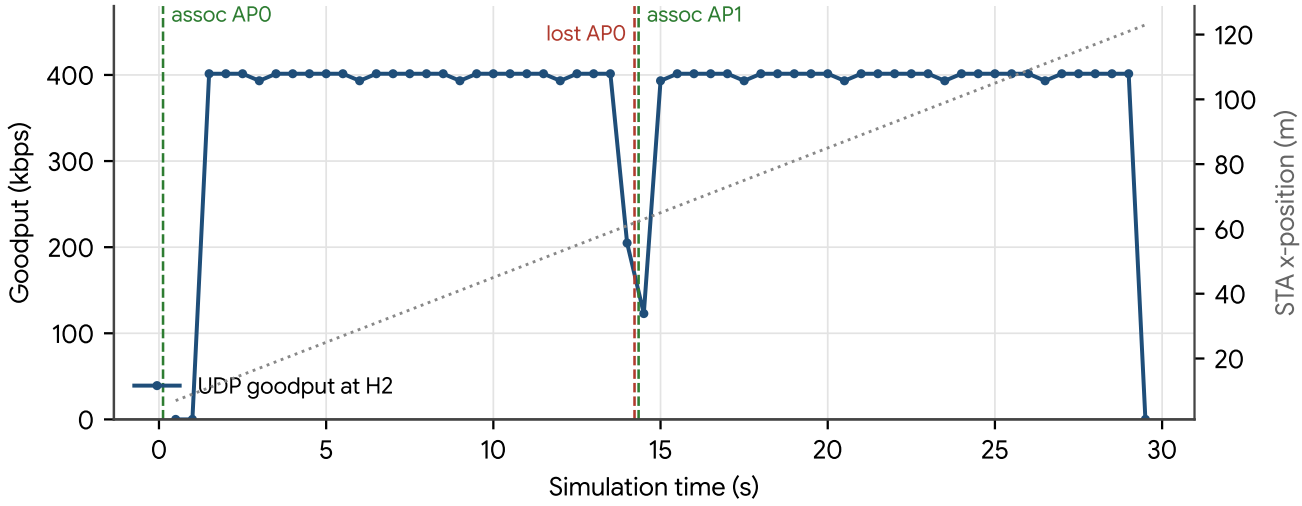
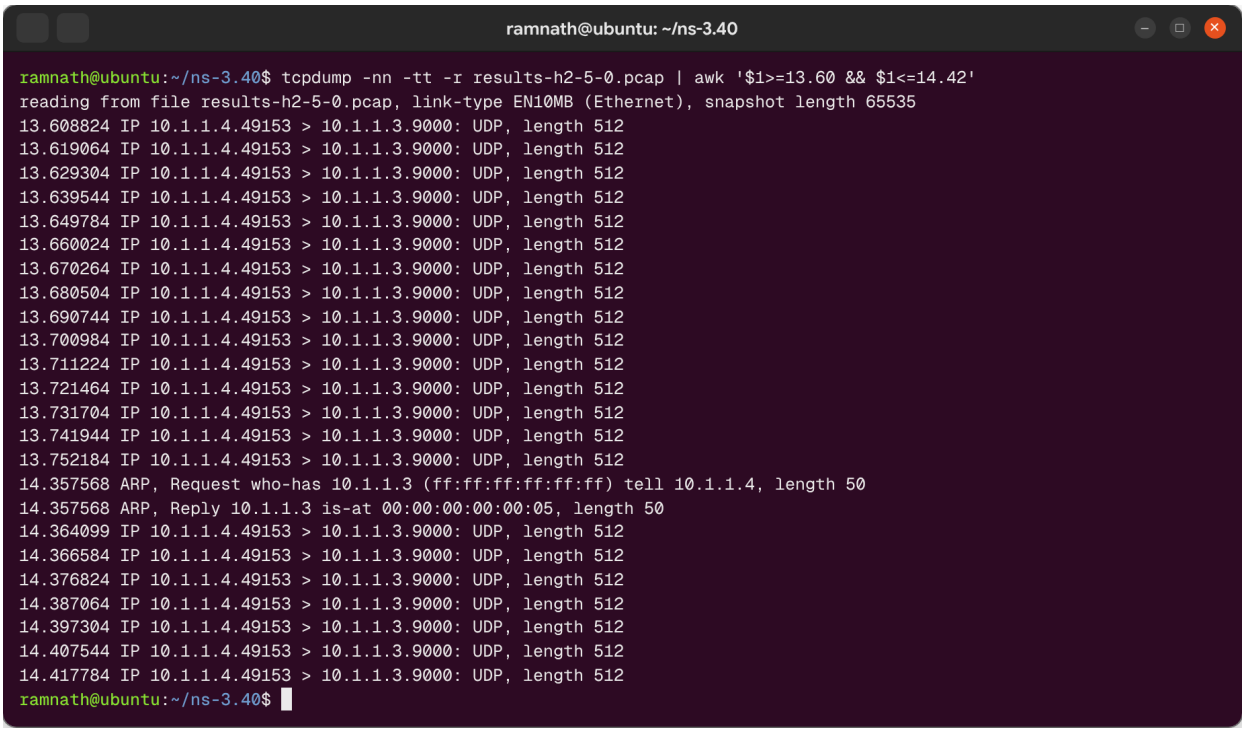


Figure 3. UDP goodput at H2.



Screenshot 5. Packets at H2 during handover.

Metric	Value
Packets sent / received	2735 / 2677
Packets lost	58 (2.12 %)
Throughput	413.0 kbps
Mean delay	3.39 ms

Table 1. FlowMonitor results.

8. Observations

- At t = 1.0 s the first ARP and UDP packets cause one `PACKET_IN` per switch. Each switch gets 3 flows; after that the controller is idle.
- STA uses AP0 until 14.22 s, then associates with AP1 at 14.34 s.
- After handover, traffic enters at S1 instead of S0. The new in_port misses the old flows, so each switch gets 2 new flows (5 total). Old flows stay.
- H2 sees no packets from 13.75 s to 14.36 s (0.61 s). This causes all 58 losses. No loss in the SDN switches.
- Flooding needs no location tracking, but every packet reaches every switch and host.
- The built-in `OFPP_FLOOD` in ns-3 sends packets back out the ingress port (byte-order bug), which loops the triangle. The controller lists output ports explicitly instead.

9. Result

The STA's UDP traffic reached H2 through the OpenFlow switches before and after handover. 2677 of 2735 packets were delivered; all losses happened during the 0.61 s Wi-Fi handover.

Student's signature

Faculty's signature